



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica



5 de mayo de 2026

Alejandro, Pablo Martin
98021

Prada, Matias Leonel
99397

Poli Salvador, Ignacio Manuel
110399

1. Consideraciones

En este trabajo practico, vamos a estar resolviendo el problema de Scaloni para definir los días que deba entrenarse y los días que convenga descansar de tal forma de tener la mayor ganancia posible para los posibles N entrenamientos.

Para realizarlo, definimos la siguiente nomenclatura:

- El esfuerzo que demanda el día i -ésimo es e_i , a su vez, esta es la ganancia máxima para ese día.
- La energía por día consecutivo disponible en el día i -ésimo viene definida por s_i
- Para cada día s_i (con $1 \leq i \leq N$) : $s_1 \geq s_2 \geq s_3 \geq \dots \geq s_N$.
- Tenemos N días de de esfuerzo al igual que tenemos N días de entrenamiento consecutivos.

Ejemplo de Resolución

2. Análisis del Problema

El problema presenta una serie de decisiones secuenciales donde cada día i el equipo técnico debe elegir entre dos opciones mutuamente excluyentes: **entrenar** o **descansar**.

Sabemos que la ganancia de un entrenamiento no es estática, sino que depende directamente de la fatiga acumulada del plantel. Podemos expresar la ganancia de entrenar un día específico j , habiendo entrenado ininterrumpidamente desde un día i (es decir, llevando $j - i + 1$ días de fatiga), mediante la siguiente expresión matemática:

$$g(i, j) = \text{mín}(e_j, s_{j-i+1}) \quad (1)$$

en donde e_j es la expectativa de esfuerzo del día j , y s representa el arreglo de energía disponible del equipo, el cual decrece a medida que aumentan los días consecutivos de entrenamiento.

2.1. Matriz de Ganancias

Para evitar recalcular las sumatorias de energía repetidas veces y aislar el concepto de “días consecutivos”, introducimos una matriz de ganancias $\text{Ganancias}(i, j)$. Esta estructura almacena la ganancia total obtenida si el equipo entrena de forma ininterrumpida desde el día i hasta el día j .

Debido a que la ganancia es acumulativa, podemos definir la construcción de esta matriz de manera iterativa:

$$\text{Ganancias}(i, j) = \begin{cases} 0 & \text{si } j < i \\ \text{Ganancias}(i, j-1) + \text{mín}(e_j, s_{j-i+1}) & \text{si } j \geq i \end{cases} \quad (2)$$

Esta precomputación resulta en una matriz triangular superior que se puede construir en tiempo $\mathcal{O}(n^2)$, y nos permite consultar la ganancia de cualquier racha de entrenamientos en tiempo constante $\mathcal{O}(1)$.

2.2. Ejemplo de la Matriz de Ganancias

Para ilustrar la construcción y el significado de la matriz $\text{Ganancias}(i, j)$, consideremos un caso de ejemplo con $n = 4$ días, donde los arreglos de esfuerzo esperado y energía disponible son:

- Esfuerzo (e): [10, 5, 12, 9]
- Energía (s): [15, 7, 5, 2]

En la estructura de esta matriz:

- Las **filas** (i) representan el **día de inicio** del bloque de entrenamiento ininterrumpido.
- Las **columnas** (j) representan el **día de finalización** de ese mismo bloque.
- Cada **celda** contiene la ganancia total acumulada si el equipo sale a entrenar todos los días en ese rango $[i, j]$.

Calculamos los primeros valores paso a paso aplicando la ecuación (2):

- **Fila 1 (Entrenamientos que empiezan el Día 1):** Arrancamos con la energía al máximo ($s_1 = 15$).

- $j = 1$ (Entrenar solo D1): $\min(e_1, s_1) = \min(10, 15) = 10$
 - $j = 2$ (Entrenar D1 y D2): $\text{Ganancias}(1, 1) + \min(e_2, s_2) = 10 + \min(5, 7) = 10 + 5 = 15$
 - $j = 3$ (Entrenar D1 al D3): $\text{Ganancias}(1, 2) + \min(e_3, s_3) = 15 + \min(12, 5) = 15 + 5 = 20$
- **Fila 2 (Entrenamientos que empiezan el Día 2):** Como asumimos que empezó el Día 2, la fatiga se resetea y usamos s_1 para el primer día de este bloque.
- $j = 2$ (Entrenar solo D2): $\min(e_2, s_1) = \min(5, 15) = 5$
 - $j = 3$ (Entrenar D2 y D3): $\text{Ganancias}(2, 2) + \min(e_3, s_2) = 5 + \min(12, 7) = 5 + 7 = 12$

Al completar todos los cálculos, la matriz resultante queda de la siguiente manera:

Inicio (i) \ Fin (j)	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	10	15	20	22
$i = 2$	-	5	12	17
$i = 3$	-	-	12	19
$i = 4$	-	-	-	9

Cuadro 1: Matriz de Ganancias precomputada para $n = 4$.

Por ejemplo, si observamos la celda en la fila $i = 2$ y columna $j = 4$, el valor 17 representa explícitamente la ganancia obtenida si el equipo decide descansar el día 1, y luego entrena ininterrumpidamente durante los días 2, 3 y 4. Las celdas vacías debajo de la diagonal no son válidas matemáticamente, ya que un bloque de entrenamiento no puede terminar en un día anterior al que empezó.

2.3. Ecuación de Recurrencia (1D)

Con la matriz de ganancias definida, podemos modelar el problema principal utilizando Programación Dinámica. Definimos nuestro arreglo unidimensional de subproblemas $OPT(i)$ como “la ganancia máxima posible considerando únicamente los primeros i días del torneo”.

Para entender la lógica detrás de la ecuación, imaginemos que estamos parados en el día i y queremos evaluar cómo maximizar nuestra ganancia acumulada hasta hoy. En lugar de preguntarnos si hoy entrenamos o descansamos, el algoritmo cambia el enfoque y se pregunta: **¿De qué tamaño será la racha ininterrumpida de entrenamientos que finaliza exactamente hoy?**

Si llamamos k a la longitud de esa racha final de entrenamientos, el algoritmo evalúa exhaustivamente todas las opciones físicamente posibles (desde $k = 1$ hasta $k = i$) y se queda con la que otorgue el mayor puntaje. Al evaluar cada k , el problema se divide en dos escenarios:

- **Escenario de Racha Total ($k = i$):** Significa que la decisión a evaluar es si conviene que el equipo haya entrenado todos los días de corrido desde que empezó el torneo hasta el día de hoy, sin descansar nunca. Como no hay días previos, la ganancia de esta opción es directamente el bloque completo que ya precomputamos: $\text{Ganancias}(1, i)$.
- **Escenario de Racha Parcial ($1 \leq k < i$):** Significa que el equipo entrena los últimos k días consecutivos (desde el día $i - k + 1$ hasta el día i), pero **obligatoriamente descansó el día inmediatamente anterior** (el día $i - k$).

Al asumir que el día $i - k$ fue un día de descanso, sabemos que la fatiga del equipo se reseteó a cero. Por lo tanto, el puntaje total de esta opción será la suma de la ganancia de la nueva racha actual de k días, más la **mejor historia posible** que el equipo haya podido construir en los días previos a ese descanso.

Esa “mejor historia pasada” (que abarca desde el día 1 hasta el día $i - k - 1$) ya fue resuelta por el algoritmo en iteraciones anteriores y está guardada en $OPT(i - k - 1)$.

Al unificar esta lógica matemática, formalizamos la ecuación evaluando de forma independiente la racha total para evitar acceder a índices negativos o inválidos en el arreglo OPT :

$$OPT(i) = \max \begin{cases} \text{Ganancias}(1, i) & \text{(Racha total sin descansos)} \\ \max_{k \in \{1, \dots, i-1\}} \left(OPT(i-k-1) + \text{Ganancias}(i-k+1, i) \right) & \text{(Rachas parciales con descanso previo)} \end{cases}$$

2.4. Ejemplo del Cálculo de Óptimos (OPT)

Continuando con nuestro ejemplo de $n = 4$ días y utilizando la matriz de la Tabla 1, evaluaremos el arreglo de óptimos OPT aplicando la ecuación (3) de forma ascendente (*bottom-up*), recordando que asumimos $OPT(0) = 0$.

- **Día 1** ($i = 1$): La única opción válida es entrenar desde el inicio ($k = 1$).
 - $OPT(1) = \text{Ganancias}(1, 1) = 10$. (*Guardamos* $k = 1$).
- **Día 2** ($i = 2$): Evaluamos los posibles cortes k :
 - Entrenar D1 y D2 ($k = 2$): $\text{Ganancias}(1, 2) = 15$.
 - Descansar D1, entrenar D2 ($k = 1$): $OPT(0) + \text{Ganancias}(2, 2) = 0 + 5 = 5$.
 - El máximo es **15**. (*Guardamos* $k = 2$).
- **Día 3** ($i = 3$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D3 ($k = 3$): $\text{Ganancias}(1, 3) = 20$.
 - Descansar D1, entrenar D2 y D3 ($k = 2$): $OPT(0) + \text{Ganancias}(2, 3) = 0 + 12 = 12$.
 - Descansar D2, entrenar D3 ($k = 1$): $OPT(1) + \text{Ganancias}(3, 3) = 10 + 12 = 22$.
 - El máximo es **22**. (*Guardamos* $k = 1$).
- **Día 4** ($i = 4$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D4 ($k = 4$): $\text{Ganancias}(1, 4) = 22$.
 - Descansar D1, entrenar D2 al D4 ($k = 3$): $OPT(0) + \text{Ganancias}(2, 4) = 0 + 17 = 17$.
 - Descansar D2, entrenar D3 y D4 ($k = 2$): $OPT(1) + \text{Ganancias}(3, 4) = 10 + 19 = 29$.
 - Descansar D3, entrenar D4 ($k = 1$): $OPT(2) + \text{Ganancias}(4, 4) = 15 + 9 = 24$.
 - El máximo absoluto del problema es **29**. (*Guardamos* $k = 2$).

El arreglo final resulta ser $OPT = [10, 15, 22, 29]$. El valor $OPT(4) = 29$ representa la máxima ganancia posible para todo el torneo.

2.5. Complejidad y Reconstrucción

La solución planteada requiere dos fases. En primer lugar, la construcción de la matriz $\text{Ganancias}(i, j)$ toma tiempo $\mathcal{O}(n^2)$ y espacio $\mathcal{O}(n^2)$.

En segundo lugar, el cálculo del arreglo unidimensional OPT de tamaño n requiere iterar i desde 1 hasta n , y en el peor de los casos, el bucle interno de maximización evalúa $i - 1$ cortes posibles. Esto resulta en una sumatoria de Gauss, la cual también se acota en tiempo $\mathcal{O}(n^2)$.

El resultado óptimo del problema se encontrará en la última posición del arreglo, $OPT(n)$.

Para reducir la complejidad de reconstruir el cronograma de entrenamientos (camino óptimo), se va a almacenar el valor del k final para poder reconstruir cada paso en $\mathcal{O}(1)$, mejorando la reconstrucción a $\mathcal{O}(n)$. Basta con realizar un proceso de *backtracking* desde $OPT(n)$, con el último k obtenido, marcando esos k días como entrenamiento, el día anterior como descanso ($i - k$), y saltando recursivamente al estado del día anterior $OPT(i - k - 1)$, así sucesivamente hasta llegar al inicio (top-down).

3. Solución propuesta

Luego de lo mencionado anteriormente presentamos un pseudocódigo que muestra la solución propuesta por programación dinámica para este problema

```
1 Entrenamientos(e,s):
2
3 Sea N igual al largo de e o s mas 1
4 Sea g una matriz de NxN con todos ceros
5 Sea k un entero iniciado en 0
6 Sea opt un arreglo de largo N inicializado con 0s
7
8 Por cada i entre 1 a N:
9   Por cada j entre 1 a N:
10    Si i>j:
11      continuamos
12    Si j == i:
13      g[i][j]=e[i]
14    Sino:
15      g[i][j]= g[i][j-1] + min(e[j],s[j-i])
16
17 Por cada i entre 1 a N:
18   Sea opt_k un arreglo de largo N inicializado en 0s
19   Sea mayor_opt un entero inicializado en 0
20
21   Por cada j entre 1 a i:
22     Si j != i:
23       opt_k[j] = opt[i-j-1]+g[i-j+1][i]
24     Sino:
25       opt_k[j] = g[1][j]
26
27     Si opt_k[j]>=mayor_opt:
28       mayor_opt = opt_k[j]
29       k=j
30   opt[i] = opt_k[k]
31
32 devolver reconstruccion(opt, k)
33
```

Figura 3.1: Pseudocódigo propuesto para resolver el problema

Siendo reconstrucción el algoritmo de para obtener la solución, que se vuelve mas simple guardando el ultimo k. Este algoritmo corresponde al siguiente pseudocódigo

```
1 Reconstruccion(opt,k):
2
3   Sea dia el calculo de len(opt)-1-k
4   Por cada i entre 0 a k
5
6
7   Sea rec una lista vacia con la soluion
8   Mientras len(rec) < len(opt):
9       Sea dia igual a len(opt)-1 -k -1
10      Por cada i entre 0 a k
11          agregamos 'E' a rec
12          agregamos 'D' a rec
13          k=dia
14
15   Damos vuelta la lista rec
16   Borramos la primera componente que es la componente 'D' agregada al final
17   Devolvemos rec
```

Figura 3.2: Pseudocódigo propuesto para reconstruir la solución

4. Demostración de Optimalidad

Para demostrar que el algoritmo de Programación Dinámica propuesto es óptimo, debemos verificar que el problema cumple con la **Subestructura Óptima** y luego proceder con una prueba por inducción sobre la validez de la ecuación de recurrencia.

El problema de la Scaloneta posee subestructura óptima si una solución óptima para un torneo de n días contiene dentro de sí soluciones óptimas para sub-torneos de $i < n$ días.

Prueba por contradicción: Supongamos que la solución óptima para n días implica que el último descanso fue en el día $i - k$, y por lo tanto el equipo entrenó desde el día $i - k + 1$ hasta el día n . Si la solución que el algoritmo eligió para los días anteriores al descanso (1 a $i - k - 1$) no fuera la óptima ($OPT(i - k - 1)$), entonces podríamos reemplazar esa porción por una que sí lo fuera, aumentando la ganancia total del torneo de n días. Esto contradice nuestra premisa de que la solución original era óptima. Por lo tanto, cualquier solución global óptima debe estar compuesta por subsoluciones óptimas.

4.1. Prueba por Inducción

Queremos demostrar que $OPT(i)$ calcula efectivamente la ganancia máxima acumulada hasta el día i .

Caso Base: Para $i = 0$, $OPT(0) = 0$. Es trivialmente óptimo ya que sin días de torneo no hay ganancia posible. Para $i = 1$, el algoritmo evalúa entrenar el primer día ($Ganancias(1,1)$). Al ser la única opción de entrenamiento, es el máximo.

Hipótesis Inductiva: Supongamos que para todo $j < i$, el valor de $OPT(j)$ es la ganancia máxima posible que se puede obtener en un torneo de j días.

Paso Inductivo: Consideremos el cálculo de $OPT(i)$. En un torneo que dura i días, en el último día (i) el equipo solo tiene dos estados finales posibles: o está entrenando o está descansando.

1. **Estado de entrenamiento continuo:** El equipo pudo haber entrenado todos los días desde el inicio ($k = i$). Esta ganancia está representada por $Ganancias(1,i)$.
2. **Estado tras un descanso:** Si el equipo no entrenó todos los días, debió existir un **último día de descanso** en alguna posición $i - k$. Para que este descanso sea el último, el equipo

necesariamente entrenó desde el día $i - k + 1$ hasta el día i (un bloque de entrenamiento de tamaño k).

La ganancia total en este segundo caso sería la suma de:

- La ganancia del último bloque de entrenamiento: $Ganancias(i - k + 1, i)$.
- La ganancia acumulada hasta antes del descanso: $OPT(i - k - 1)$. Notar que el día $i - k$ no suma ganancia por ser de descanso.

Dado que el tamaño del último bloque de entrenamiento k puede ser cualquier valor entre 1 y $i - 1$, la ecuación de recurrencia:

$$OPT(i) = \max \left(Ganancias(1, i), \max_{1 \leq k < i} \{OPT(i - k - 1) + Ganancias(i - k + 1, i)\} \right)$$

evalúa todas las formas posibles, exhaustivas y mutuamente excluyentes, de terminar el día i . Como por Hipótesis Inductiva todos los valores $OPT(i - k - 1)$ ya son los máximos para sus respectivos días, y estamos eligiendo el máximo entre todas las particiones válidas de la solución, $OPT(i)$ es necesariamente la ganancia máxima para i días.

5. Análisis de Variabilidad y Optimalidad

A continuación, analizamos cómo la variabilidad de los parámetros de entrada afecta el comportamiento del algoritmo de Programación Dinámica, la optimalidad de la solución teórica y la posterior reconstrucción empírica del cronograma.

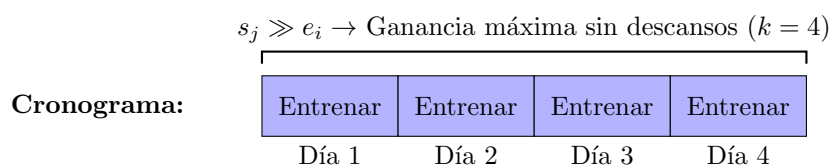
5.1. Afectación de la variabilidad a la ganancia y optimalidad

La variabilidad en las distribuciones de los valores de esfuerzo (e) y energía (s) **no afecta la optimalidad de la solución algorítmica**. Al evaluar exhaustivamente todos los tamaños de bloques posibles k hacia atrás y apoyarse en subestructuras óptimas ya calculadas, la ecuación de recurrencia garantiza matemáticamente encontrar siempre la ganancia máxima absoluta, sin importar si los datos son uniformes, extremos o atípicos.

Sin embargo, la forma de estos arreglos **sí afecta drásticamente la estructura del cronograma óptimo resultante** y la cantidad de soluciones óptimas posibles. Analizamos en detalle distintos escenarios:

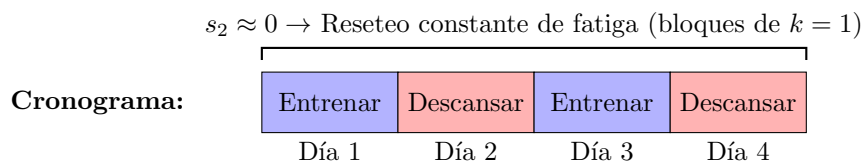
1. Caso A: Energía casi inagotable o constante.

- **Descripción:** El arreglo de energía s se mantiene en valores muy altos y decae sumamente lento, superando holgadamente a los valores de esfuerzo e_i en la gran mayoría de los días.
- **Análisis:** En este escenario, la función de ganancia diaria $\min(e_j, s_{j-i+1})$ estará dominada exclusivamente por los valores de e_j . Dado que la energía no es un limitante, tomar un día de descanso implica sumar 0 a la ganancia de ese día sin obtener un beneficio real a cambio para el futuro. Como resultado, el algoritmo tenderá a elegir bloques de tamaño máximo ($k = i$), construyendo un cronograma de entrenamiento continuo y sin interrupciones.



2. Caso B: Caída abrupta de energía.

- **Descripción:** El equipo sufre de fatiga extrema muy rápido. Por ejemplo, la energía inicial s_1 tiene un valor alto, pero para el segundo día consecutivo (s_2) decae abruptamente a valores cercanos a 0.
- **Análisis:** Si la energía se desploma al segundo día de entrenamiento, cualquier bloque de tamaño $k \geq 2$ sumará un valor marginal en sus últimos días. Al algoritmo le resultará matemáticamente mucho más rentable insertar un día de descanso para resetear la fatiga y volver a aprovechar el alto valor de s_1 . En consecuencia, la estructura del cronograma será una secuencia fragmentada, alternando estrictamente días de entrenamiento cortos (bloques de $k = 1$) con días de descanso.



5.2. Comportamiento en Programación Dinámica (Bottom-Up) vs. Reconstrucción (Top-Down)

En contextos de alta variabilidad que generan empates sistemáticos (como en el Caso C), el impacto se debe interpretar de manera distinta según la fase del algoritmo:

- **Fase Bottom-Up (Llenado del arreglo OPT):** La variabilidad y la multiplicidad de opciones **no afectan en lo absoluto el cálculo del valor óptimo**. Cuando la función máx de la ecuación de recurrencia evalúa diferentes tamaños de bloque k y descubre que varios devuelven el mismo puntaje máximo, la ecuación simplemente asimila ese número. El valor final acumulado que llegará a $OPT(n)$ será único, robusto y representará la ganancia teórica inamovible, a prueba de cualquier volatilidad en los datos de entrada.
- **Fase Top-Down (Algoritmo de Reconstrucción):** Aquí es donde la alta variabilidad tiene un impacto real y directo. Si durante la fase *Bottom-Up* existieron múltiples valores de k que empataron como el máximo para un mismo día i , significa que existen **múltiples cronogramas estructuralmente distintos que logran exactamente la misma ganancia total**.

Un algoritmo de reconstrucción típico almacena solo un valor de k ganador por celda para lograr una complejidad de $\mathcal{O}(n)$. La decisión de *cuál* k se guarda ante un empate depende de un simple operador lógico en la implementación: si el código utiliza un comparador estricto ($>$), conservará el primer k que encuentre; si utiliza mayor o igual ($\geq$), sobrescribirá la variable y se quedará con el último k óptimo evaluado.

Por lo tanto, ante datos que generan empates constantes, el cronograma específico devuelto (el detalle de qué días puntuales se entrena o se descansa) puede variar diametralmente de una ejecución a otra o entre distintas implementaciones del mismo algoritmo, **aunque todas esas versiones representen caminos 100 % óptimos**.

6. Mediciones

Como sabemos el código corre en un tiempo $\mathcal{O}(n)$ pero para poder demostrar que el código en realidad demora dicho tiempo, vamos a estar realizando una medición sobre el algoritmo.

El método de medición consiste en realizar un set de datos usando un código que genera una cantidad de datos igual al parámetro de entrada, es decir, si el parámetro de entrada es 10, va a

generar un archivo *10.txt* con 10 rivales. Si se pasa una lista de M valores, genera M archivo con dichos valores (el parámetro esta hardcoded en el archivo python).

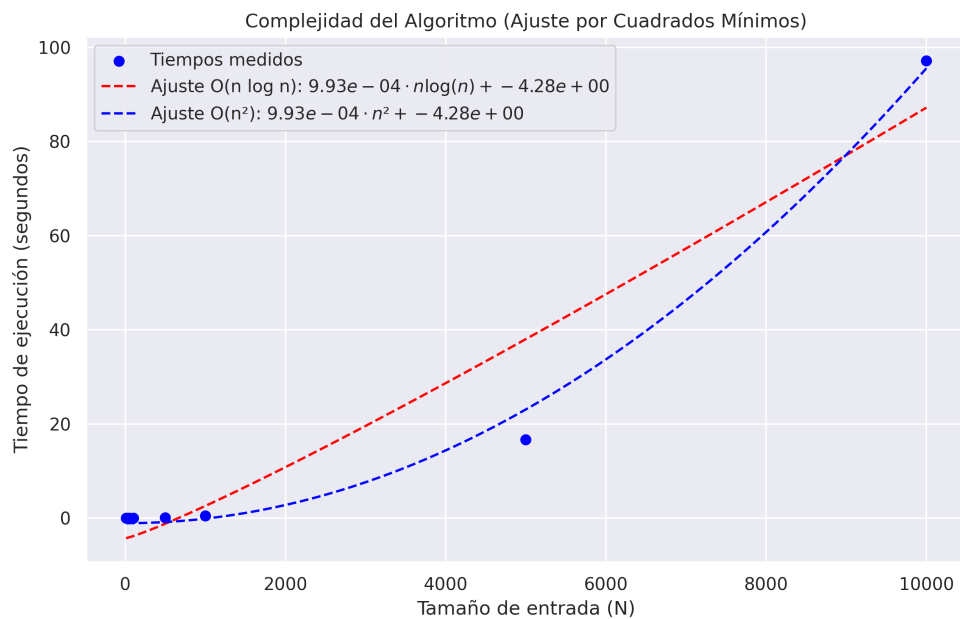
Luego, corremos el código en modo de medición usando el comando `python tp2.py -medir` (tener cuidado con el `-` de latex, puede fallar al copiarlo directamente), el cual realiza la medición de los tiempos por cada uno de los archivos generados anteriormente y guarda el resultado en un archivo llamado *mediciones.txt* con el siguiente formato:

```
1 10,tiempo de n=10
2 100,tiempo de n=100
3 1000,tiempo de n=1000
4 ....
5 N,tiempo de n=N
```

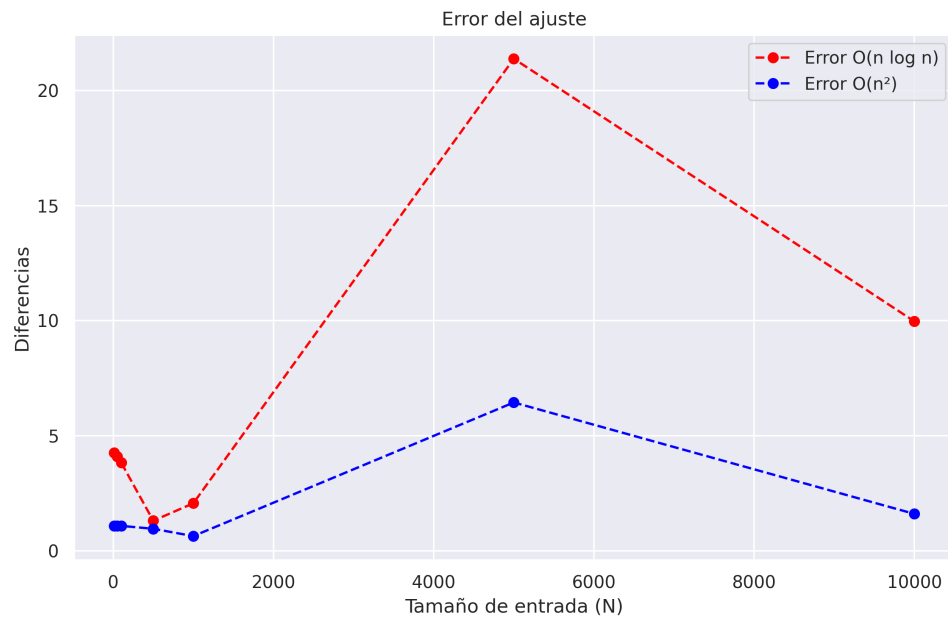
Figura 6.1: Archivo mediciones.txt

en donde, cada fila indica el tiempo que demoro el algoritmo con la cantidad de N datos en el archivo. De esta forma vamos a poder graficar los resultados y aproximar dicho puntos usando el método de cuadrado mínimo.

La siguiente figura muestra la aproximación de las funciones $O(n \log n)$ y $O(n^2)$ viendo que los puntos se aproximan mejor a una función de $O(n)$ que al tiempo de $O(n \log n)$, demostrando que la teoría es correcta.



Es importante mostrar el error de la función aproximada con el teórico de esta forma podemos demostrar de una forma mas teórica a que función corresponde mas, demostrando que de forma teórica el error de la aproximación de $O(n)$ es menor al error de $O(n \log n)$. Esto lo observamos en la siguiente figura.



7. Conclusiones

Para encontrar la solución del problema, es importante destacar que en programación dinámica es necesario realizar un análisis previo del problema, siendo este el mayor problema del mismo. Teniendo en cuenta los algoritmos de Greedy que se tiene que elegir la decisión codiciosa que nos puede dar la mejor solución local en el estado correspondiente, en programación dinámica buscamos encontrar la mejor solución del estado actual definiendo mis variables de estados, de esta forma puedo guardar o memorizar los valores previos para poder ahorrar re calcular el mismo estado varias veces, además la programación dinámica nos permite explorar diferentes caminos para poder encontrar la mejor solución.

Es importante que reconstruir la solución tiene su complejidad que tenemos que asegurar que el tiempo que demore a lo sumo sea igual que encontrar la solución, en el caso que sea peor, sería mejor reconstruir el algoritmo porque encontrar la solución debería ser el algoritmo con mayor complejidad del problema.

Un punto extra es la medición de los tiempos del algoritmo, esto nos permite chequear que la teoría se corrobore con la práctica realizada, usando métodos matemáticos que nos permiten buscar una función que mejor aproxima los puntos medidos desde un código aparte.